

Build a Vision-Language-Action Robotic System for Natural Language Object Manipulation

close

Problem Statement

The goal is to develop a robotic system capable of understanding and executing object manipulation tasks based on natural language commands. This system should integrate perception, language understanding, and action execution to interpret and act on instructions like "Move the blue block to the right of the green cube." The system will be trained and evaluated using publicly available datasets to ensure robustness and generalization across diverse tasks and environments. The focus is on building a pipeline that can parse language, detect objects in the scene, and generate precise robotic actions for manipulation.

Key Expectations & Deliverables

Deliverables

1. Code Repository: A fully functional Python/PyTorch implementation of the robotic system.
2. Demo Video: A video demonstrating the system executing 5+ different natural language commands in simulation.
3. Final Report: A comprehensive report detailing methodology, challenges, and performance improvements.
4. Tools & Libraries: Vision (OpenCV, Detectron2, or OpenVLA), Language (Hugging Face Transformers BERT/GPT or Llama 2 for parsing and generating language-based instructions), Simulation (PyBullet or ROS for simulating robotic actions and environments).

Expected Capabilities

- Natural Language Command Interpretation: The system should be able to understand and act on complex instructions involving object attributes (eg: color, position) and actions (eg: grasp, move).
- Multi-Object Manipulation: The robot should be able to identify and manipulate multiple objects in a scene based on language input.
- Generalization: The system should demonstrate generalization to new tasks and unseen objects by leveraging pretraining on diverse datasets.

Definitive Target KPIs & Benchmarks

Metric	Target	Benchmark
Task Success Rate (TSR)	80–85%	70–75% (SOTA models like RT-1-X or Octo)
Goal Condition Accuracy (GCA)	90%	80–85% (based on prior VLA benchmarks)
Command Interpretation Accuracy	85%	75–80% (based on language grounding models like CLIPort)
Task Completion Rate (TCR)	80%	65–70% (based on imitation learning baselines)
Error Analysis Coverage	100%	N/A (required for all deliverables)

These KPIs are aligned with state-of-the-art benchmarks in vision-language-action (VLA) models, such as OpenVLA, which has demonstrated strong generalization and task success in real-world and simulated environments. The proposed system should aim to match or exceed these benchmarks.

Suggested Open Models and Datasets

Vision & Language Models

- OpenVLA: A 7B parameter open-source VLA model supporting multiple robot platforms and can be fine-tuned for new tasks.
- Detectron2: For object detection and scene understanding.
- Hugging Face Transformers (BERT/GPT): For parsing and understanding natural language instructions.

Datasets

- Open X-Embodiment: Large-scale dataset of robot manipulation trajectories.
- COCO (Common Objects in Context): Useful for object detection training and scene understanding.
- Something-Something V2: Provides action recognition data for understanding verbs in language commands (eg: push left, lift slowly).
- RoboNet: Offers robotic control simulations for grasping and pushing tasks.
- ALFRED: Language-guided task sequences for task planning and execution.
- VLA-3D: A dataset for 3D scene graphs with spatio-semantic relations, useful for grounding language in complex environments.

- LIBERO: A dataset for long-horizon robotic manipulation tasks, particularly useful for cloth and deformable object handling.

Simulation & Execution Tools

- PyBullet: For simulating robotic actions and physics-based interactions.
- ROS (Robotic Operating System): For integrating perception, planning and control modules in a unified framework.
- SigLIP and DinoV2: Used as visual backbones in OpenVLA for image patch embeddings.

02

Context-Aware Flow Embeddings for Adaptive AI based Network Traffic Classification

close

Problem Statement

Traditional network traffic classification methods are ineffective in dynamic environments and for new traffic types due to their reliance on deep packet inspection (DPI) and static rule-based systems. These approaches struggle with encrypted traffic and evolving network conditions. The goal is to develop a packet flow encoder that generates context-aware embeddings using flow-level features and network characteristics (e.g., RTT, jitter) to enable accurate traffic classification in real-time and under changing conditions. For instance, a video streaming flow should be embedded closer to other streaming flows than to gaming traffic. This requires a system that adapts to emerging traffic types (eg: XR), and generalizes across diverse network scenarios.

Key Expectations & Deliverables

- Encoder Model: Design a neural network architecture (e.g., LSTM, Transformer, or CRNN) that combines flow features (5-tuple, packet rate, inter-arrival times) with network characteristics (RTT, jitter) to produce embedding vectors.
- Contrastive Learning: Implement a contrastive loss function to train the encoder, ensuring that similar traffic types (eg: Youtube and Netflix) are

embedded closely, while dissimilar types (eg: videos streaming vs gaming) are separated.

- Classifier Integration: Use the embeddings to train a downstream classifier (e.g., k-NN, SVM) that can identify traffic types with high accuracy.

Definitive Target KPIs & Benchmarks

- Embedding Similarity: Cosine similarity of > 0.7 for intra-class traffic (eg: Youtube and Netflix) and < 0.3 for inter-class traffic (eg: videos streaming vs gaming).
- Classification Accuracy: At least 90% accuracy on a test dataset of encrypted and unencrypted traffic types.
- Generalization: Demonstrate the model's ability to generalize to new, unseen traffic types with at least 85% accuracy in cross-validation tests.
- Real-Time Performance: Ensure the model can process and classify traffic in < 100 ms per flow for real-time applications.

Suggested Open Models & Datasets

- 5G Traffic Datasets: <https://www.kaggle.com/datasets/kimdaegyeom/5g-traffic-datasets>
- CESNET-QUIC22: A dataset with encrypted QUIC traffic, including packet sizes, inter-packet times, and SNI domains, suitable for training and testing the encoder.
- MAWI Dataset: A real-world traffic dataset with diverse flow patterns and encrypted traffic, ideal for evaluating generalization and robustness.
- PacketCLIP: A multi-modal embedding framework that aligns packet data with natural language descriptions, used for interpretability and semantic reasoning.
- Generating a dataset using Manual Packet Capture (eg: Running Youtube/Netflix in good/bad network conditions).

03

Context-Aware, Adaptive Memory Solution for Mobile Agentic Systems

close

Problem Statement

Modern on-device (smartphone or edge device) agentic systems are expected to handle increasingly complex and context-sensitive tasks, from real-time GenAI inference to multitasking across multiple applications. However, the limited memory resources of such devices often lead to suboptimal performance, increased latency, and reduced user experience. The core challenge lies in efficiently managing memory to adapt to the dynamic nature of user activities and anticipated future tasks. This includes intelligently allocating memory to foreground and background applications, predicting the next user context to pre-load relevant components, and managing caching to avoid thrashing and excessive swapping.

A context-aware memory solution is essential to ensure that these systems remain responsive, efficient, and stable under diverse and demanding conditions. Without such a solution, memory bottlenecks can severely limit the capability of on-device AI agents, especially in resource constrained environments like smartphones and edge devices.

Key Expectations & Deliverables

The team is expected to deliver a context-aware, adaptive memory system that demonstrates the following capabilities:

- Context-Aware Memory Allocation: A memory manager that dynamically allocates resources based on the user's current and predicted context. This includes prioritizing memory for critical applications and adjusting allocations in real-time.
- Predictive Pre-Loading: Integration of a machine learning model or heuristic that predicts the user's next likely application or task and pre-loads relevant data to reduce latency.
- Adaptive Caching Policies: Implementation of a caching mechanism that adjusts retention and eviction of cached applications based on usage patterns and memory availability.

Definitive Target KPIs & Benchmarks

KPI	Target	Benchmark
Application Load Time Improvement	20%	Baseline (no memory optimization)
App Launch Time Improvement	10%	Baseline (no memory optimization)
Memory Thrashing Reduction	50%	Baseline (no memory optimization)
System Stability	0 stability issues	Baseline (no memory optimization)

Accuracy of Next Context Prediction	≥75%	Random prediction baseline
Caching Hit Rate	≥85%	Static caching baseline
Memory Utilization Efficiency	30%+ improvement	Baseline (no optimization)

These KPIs will be measured using synthetic and real-world datasets, as well as benchmarking against existing memory management systems and caching strategies.

Suggested Open Models and Datasets

ML Models for Context Prediction

- Transformer-based models: Can be used for predicting user behavior or next context based on historical data.
- Time Series Forecasting Models (e.g., Prophet, ARIMA, or LSTM networks): Useful for predicting user interaction patterns over time.
- Reinforcement Learning Models (e.g., TD3, DDPG): As used in, these models can help in optimizing memory allocation and caching decisions in real-time.

Datasets for Training and Evaluation

- Context Query Logs: This dataset includes real-world context queries inspired by parking-related traffic in Melbourne, Australia. It can be used to simulate and evaluate the system's ability to handle heterogenous and time-sensitive queries.
- Android Usage Patterns: This dataset captures user interaction with Android applications, including app switches, usage duration, and background activity. It is ideal for training predictive models for app switching and pre-loading.
- KV Cache Workloads: A set of workloads designed to stress-test memory allocation and caching strategies, especially in multi-model execution scenarios.

Designing a Robust AI System for Speech Disentanglement

close

Problem Statement

Speech disentanglement is the process of isolating individual components from a mixed audio signal. In the context of speaker-specific custom word detection, the system must identify a user-defined word spoken by a specific person, even in the presence of diverse environmental noises such as crowd babble, traffic, or background conversations. The system must not only detect the word accurately but also ensure that it is spoken by the designated speaker and not by someone else, even if the word is phonetically similar. This task requires a deep learning solution that can operate efficiently and accurately in real-time, with minimal computational resources, while maintaining high performance across different languages, accents, and distances.

Key Expectations & Deliverables

Expectations

- **Robustness:** The system must perform reliably in various noise conditions, including crowd, babble, and traffic noise, with SNR ranging from -5dB to 30dB.
- **Efficiency:** The model must be optimized for real-time processing, with a maximum execution time (xRT) of less than 0.2 seconds and a parameter count under 3 million.
- **Detection Quality:** The system must accurately detect the custom word from the designated speaker and reject phonetically similar words.
- **Generalization:** The model must generalize well to unseen data without significant retraining, ensuring adaptability to new users and environments.

Deliverables

- A fully functional speech disentanglement system (including code + model).
- A demonstration showcasing the system's performance across different audio samples, including clean and noisy environments.

Definitive Target KPIs & Benchmarks

Metric	Target
True Acceptance (TA) for Clean	≥ 99%
True Acceptance (TA) for Noisy	≥ 90%

False Acceptance (FA)	< 1/hr (1 FA in 1 hour of recorded cases)
SNR Range	-5dB to 30dB for Crowd, Babble, Traffic Noise
Distance Range	0.5 meters to 5 meters
Speaker Support	Male & Female
Model Parameters	< 3M
Execution Time (xRT)	< 0.2 seconds

These KPIs ensure that the system is both accurate and efficient, capable of operating in real-world conditions with minimal false positives and high reliability.

Suggested Open Models & Datasets

1. LibriPhrase: A dataset for keyword spotting and speaker verification, ideal for training and testing custom word detection models.
2. Google Speech Commands: A dataset containing one-second audio clips of spoken words, useful for benchmarking and evaluating keyword detection systems.
3. Voxceleb – 1 & 2: Large-scale datasets for speaker recognition and verification, providing diverse speaker samples across multiple languages and accents.
4. Zero-shot TTS Models: Can be used to generate synthetic speech data for different speakers and words, enabling training on a broader range of phonetic variations.

05

Designing Empathetic Intelligence User Experience for Everyday Life

close

Problem Statement

In the near future, Gen Z and Gen Alpha will inhabit a hyper-connected world where devices, environments, and services are continuously generating and consuming data. Despite the rapid advancements in AI, everyday life remains

fragmented. Users, especially young individuals, face cognitive overload from managing multiple tasks, apps, devices, and decision-making processes, which leads to stress, fatigue and inefficiencies.

The challenge is to design an AI-powered, empathetic experience, device and/or platform leveraging user's device ecosystem that understands, anticipates, and enables user's daily life. The platform should seamlessly integrate personal data, contextual signals, and world data to create a holistic experience that reduces cognitive load, enhances emotional well-being, enables cross-task continuity, ensures privacy and trust, and evolves with users over time.

Key Expectations & Deliverables

- **Concept Framework:** Define the data inputs, intelligence layer, and experience layer of the empathetic experience, device and/or platform. This should include how it would process and learn from user data across devices.
- **Product Concept:** Present clear use cases and user journeys for diverse user-scenarios. These scenarios should reflect real-life challenges such as friction, systemic-conflicts, and micro-decision fatigue.
- **Experience Design:** Develop multi-device flows and UI/UX prototypes that illustrate how the solution will interact with users across different devices and environments.
- **Technical Approach:** Outline a proof of concept (POC), high-level architecture, and AI strategy for the platform. This can include the integration of Samsung's existing ecosystem and AI technologies
- **Impact Evaluation:** Establish metrics for success and scalability, including both qualitative and quantitative evaluations.
- **KPIs & Benchmarks Report:** Test proposed solution with 8-10 users for qualitative research, and 30-50 users for quantitative survey. Provide a detailed KPI report based on user feedback, focussing on experience of task completion, quality and AI autonomy.

Definitive Target KPIs & Benchmarks

Evaluate the proposed solution through user testing, surveys, and behavioural analytics to ensure it meets the needs of GenZ and Gen Alpha users.

The success of the empathetic experience, device and/or platform will be measured against the following KPIs:

- Effort Reduction: The solution should reduce the time or steps required to complete a task by at least 30% compared to current solutions.
- Task Completion: The platform should ensure that 90% of user tasks are completed successfully without errors.
- AI Autonomy: Evaluate the relevance and quality of AI recommendations and/or autonomous actions. Target: $\geq 85\%$
- Experience Quality & Emotional Impact: At least $\geq 4.5/5$ user satisfaction score and measurable reduction in stress indicators
- Willingness to Pay: At least 60% of target users should express willingness to pay for the platform, indicating perceived value and satisfaction.

Submissions must include verifiable baseline vs prototype validation with raw evidence (recordings/screenshots/surveys data) and clearly defined measurement methods; emphasis will be placed on credibility over absolute performance.

06

Enhancing Reasoning in Small Language Models (SLMs) using Reinforcement Learning

close

Problem Statement

Large Language Models (LLMs) demonstrate strong reasoning across domains such as mathematics, logical inference, and multi-step decision-making. However, their deployment is often constrained by latency, cost, and hardware limitations, especially in on-device or real-time applications.

Small Language Models (SLMs) ($\leq 7B$ parameters) offer a promising alternative due to their efficiency, but they significantly lag behind in:

- Multi-step reasoning (Limited capacity $\rightarrow$ weak reasoning chains)
- Logical consistency
- Planning and self-correction

Reinforcement Learning (RL) has shown strong improvements in reasoning for large models, but directly applying these techniques to SLMs is

ineffective without redesigning the pipeline. Also, SLMs have high sensitivity to reward design.

Key Expectations & Deliverables

Some directions that can be explored:

- Outcome + process-based reward design
- Lightweight / efficient reward mechanisms
- Stability techniques (KL tuning, entropy, curriculum)
- Hybrid pipelines (SFT + RL, distillation + RL)

Participants are expected to deliver:

- A RL-based training code tailored for SLMs that improves reasoning while maintaining efficiency.
- A final model with inference code for evaluation.
- Detailed approach, results, and insights gained.

Definitive Target KPIs & Benchmarks

Benchmark	Minimum Target	Expected Improvement
GSM8K	$\geq 50\%$	$\geq +5\%$ improvement over baseline model
MMLU	$\geq 45\%$	$\geq +5\%$ improvement over baseline model
StrategyQA	$\geq 65\%$	$\geq +5\%$ improvement over baseline model

Improvement should be demonstrated on at least two of the above three benchmarks.

The solution should also be efficient and effective, ensuring minimal latency overhead.

Suggested Open Models & Datasets

Participants can leverage the following open models and datasets to build and evaluate their solutions:

Models

- Qwen 2.5 7B
- Gemma 4 E4B
- Phi-3-Mini (3.8B)

Datasets

- GSM8K
- AQuA-RAT
- MMLU

Recommended Infra Requirements: 2× NVIDIA GPU with ≥ 24 GB VRAM, 12+ CPU cores, 64 GB system RAM.

07

GenAI based Ethical Deepfake Video Generation

close

Problem Statement

Deepfake video generation, powered by advanced AI models like GANs and diffusion models, has rapidly evolved from a niche innovation to a mainstream phenomenon with significant ethical and societal implications. While the technology offers legitimate applications in film, education, and accessibility, it also presents substantial risks such as misinformation, identity fraud, and reputational harm. The challenge lies in developing a system that ensures high technical fidelity while incorporating ethical safeguards, demographic and contextual diversity, and structured metadata. This requires not only advanced AI models, but also robust detection mechanisms and responsible deployment practices.

Key Expectations & Deliverables

- **Ethical Safeguards:** Implement mechanisms to ensure consent and prevent misuse, such as watermarking and metadata tagging.
- **Technical Fidelity:** Generate high-quality deepfake videos with smooth motion (Temporal Consistency ≥ 0.85) and minimal visible glitches (Artifact Rate $\leq 5\%$).
- **Demographic Diversity:** Include at least 200 unique identities, with gender balance between 40–60%, four age groups, and five ethnic groups.
- **Contextual Diversity:** Cover at least five applications (e.g., interviews, speeches, social media posts) with at least 10 videos each, ensuring no single application exceeds 30% of the dataset.

- Metadata / Description: Include structured details for each video such as Title, summary, type of deepfake, resolution, and temporal variations.

Definitive Target KPIs & Benchmarks

Deepfake Video Quality

- Temporal Consistency ≥ 0.85
- Artifact Rate $\leq 5\%$

Demographic Diversity

- Unique identities ≥ 200
- Gender balance 40–60%
- Age groups ≥ 4
- Ethnicity coverage ≥ 5

Contextual Diversity

- Application Coverage ≥ 5
- Per-application samples ≥ 10 videos each
- Platform Diversity ≥ 4 (e.g., YouTube, Instagram/Reels, TikTok, video conferencing)
- Content Variation Score ≥ 0.8 (different tones: formal, casual, emotional)
- Scenario Balance: No single application $> 30\%$ of dataset

Suggested Open Models or Datasets

- Deepfake Detection and Localization (DDL) Dataset: This dataset includes both unimodal (image) and multimodal (audio-video) data, with pixel-level annotations for manipulated regions, covering a wide range of deepfake generation methods.
- Celeb-DF v2: An expanded version of Celeb-DF, this dataset includes 6K videos with increased quantity and diversity, making it suitable for training and evaluating deepfake detection models.
- FFHQ: A high resolution facial dataset consisting of 70K face images at 1024×1024 resolution, showcasing diverse ethnicity, age, and backgrounds.
- VoxCeleb2: A dataset five times larger than VoxCeleb1 with improved racial diversity, making it ideal for training models with diverse demographic data.
- MEAD: An emotional audiovisual dataset that provides facial expression information during conversations with various emotional

labels, suitable for generating deepfake content with emotional variations.

- Deepfake Detection Challenge (DFDC) Dataset: The official dataset for The Deepfake Detection Challenge, containing a substantial amount of interference information, suitable for benchmarking detection models.
- FakeAVCeleb: A novel audio-visual multimodal deepfake detection dataset with deepfake videos generated using four forgery methods, useful for evaluating multimodal detection techniques.

08

Immersive Spatial Voice Call Experience with AI

close

Problem Statement

Traditional voice calls are limited to monaural, resulting in a flat and less engaging communication experience. The objective is to design and develop an AI-driven system that transforms these calls into immersive, spatially-aware interactions by reconstructing audio into a realistic 3D soundscape. This system should dynamically adjust audio based on user context, such as movement in a noisy environment, to enhance presence, clarity, and user engagement. The goal is to move beyond conventional communication and create a lifelike, intuitive multi-party interaction experience with improved intelligibility and natural sound propagation.

Key Expectations & Deliverables

- Immersive Audio System: Develop a system that simulates 3D spatial audio, using x, y, and z axes to represent sound in a full 3D space.
- Dynamic Audio Adjustment: Implement features that adjust audio based on user context, such as location or movement in a virtual environment.
- Real-Time Processing: Ensure the system supports real-time audio processing with an inference latency of less than 20ms and overall audio input latency of 40-60ms.
- Multi-Party Support: Enable the system to handle multiple users in a single call, with realistic spatial perception for each participant.
- Low Model Size: Design the AI model to be under 50MB for efficient deployment and performance.

Definitive Target KPIs & Benchmarks

KPI	Target	Benchmark
Audio Latency (Inference)	< 20ms	20-30ms for real-time interactive systems
Audio Latency (Input)	< 40-60ms	60-100ms for acceptable real-time audio processing
Model Size	< 50MB	50-100MB for mobile and embedded applications
Spatial Audio Accuracy	95%	90-95% for immersive audio experiences
User Engagement Score	85%	75-85% for user satisfaction in immersive systems
System Scalability	Support up to 3 concurrent users	3-4 users for real-time multiparty audio systems
UI Responsiveness	< 100ms	100-200ms for UI interactions in real-time applications

Suggested Open Models and Datasets

- MRSAudio Dataset: A large scale multimodal recorded spatial audio dataset with refined annotations.
- Virtual Acoustic Dataset: Virtual Acoustics (VA) is a real-time auralization framework for scientific research providing modules and interfaces for experiments and demonstrations.
- SONICOM HRTF Dataset: Includes different types of data measured from 200 subjects, including HRTFs, HpTFs, depth pictures and 3D models of subject' ears, heads and torsos .
- Agora Spatial Audio SDK: Offers a comprehensive solution for simulating 3D spatial audio with features like sound blur and air attenuation. It supports real-time spatial positioning and multi-platform compatibility.
- Common Voice Dataset (Mozilla): A large open-source dataset containing voice recordings in multiple languages.
- LibriSpeech Dataset: A dataset derived from audiobooks, suitable for training ASR and NLU models.

Occlusion-Aware 3D Scene Reconstruction in Partially Observable Real-World Environments

close

Problem Statement

Modern autonomous systems such as mobile robots, including indoor service robots, warehouse automation systems, and autonomous agents, rely heavily on sensor data (e.g., RGB-D cameras or LiDAR) to build 3D scene models for navigation. However, real-world environments are inherently partially observable, which leads to several critical challenges:

- Occluded obstacles: Objects hidden behind furniture or other structures are not visible, creating blind spots.
- Incomplete maps: Limited viewpoints lead to gaps in the 3D reconstruction, making free space estimation uncertain.
- Navigation risks: Missing data in the 3D mesh can result in collisions with unseen hazards.
- Holes in mesh: Missing sensor data results in incomplete or fragmented 3D mesh representations.

Traditional mapping systems assume full visibility and fail to infer or reconstruct occluded regions. This makes them unreliable in cluttered or dynamic environments. The core challenge is to distinguish between free space, occupied space, and occluded (unknown but potentially occupied) space, and to predict and fill in plausible geometry for occluded regions to improve navigation safety and efficiency.

Key Expectations & Deliverables

1. Occlusion-Aware 3D Reconstruction System: Develop a system that can infer and reconstruct occluded regions in a 3D scene. The system must generate a complete, contiguous 3D mesh from partial sensor data (RGB images & sparse depth data - around 500 depth pixels per image).
2. Classification of Scene Elements: Clearly distinguish between free space, occupied space, and occluded (unknown) space. The classification should be robust in cluttered. Further classify the 3D mesh like floor, wall, ceiling, platform & other.

- 3. Hole-Filling Algorithm: Implement an algorithm that fills in holes in the 3D mesh based on spatial reasoning and learned priors. The filled regions should be geometrically plausible and consistent with the observed parts of the scene.
- 4. Integration with Navigation Stack: Integrate the 3D reconstruction module into a navigation pipeline. The system should use the reconstructed scene for safe and efficient path planning, avoiding occluded hazards.

Definitive Target KPIs & Benchmarks

Metric	Target	Benchmark
F1 score for filled 3D mesh	> 0.95	0.85
Semantic accuracy of classes	> 90%	80%
Reconstruction accuracy	< 2 cm	5 cm

Suggested Open Models & Datasets

- ScanNet Dataset: Large-scale RGB-D dataset with 3D reconstructions of indoor scenes. useful for training and evaluating occlusion-aware reconstruction algorithms.
- Atlas - End-to-end 3D Scene Reconstruction from Posed Images: The method directly regresses a truncated signed distance fucntions (TSDF) from RGB images, eliminating the need for intermediate depth maps and improving 3D reconstruction accuracy.
- VGGT - Visual Geometry Grounded Transformer: VGGT is a feed-forward neural network that simultaneously infers key 3D attributes (camera parameters, point maps, depth maps, and 3D point tracks) from varying number of views, offering simplicity, efficiency, and state-of-the-art performance across multiple 3D tasks.
- RGB-D Dataset 7-scenes: The 7-scenes dataset is a collection of tracked RGB-D camera frames.

Problem Statement

Telecom Radio Access Networks (RANs) are growing in complexity and scale, making tasks like root cause analysis, anomaly detection, and optimization increasingly difficult to manage with traditional tools. Manual intervention by Subject Matter Experts (SMEs) is still prevalent, leading to delays, inefficiencies, and scalability issues. The goal of this hackathon is to develop a Retrieval-Augmented Generation (RAG) application tailored for telecom tasks (eg: ORAN, 3GPP spec understanding, 3GPP QnA, etc.) that automates and accelerates decision-making while maintaining domain specificity, real-time feasibility, technical efficiency, security, and explainability.

The application should support automated root cause analysis, anomaly detection, and network optimization using public datasets such as TeleQnA and O-RAN, and be evaluated on a set of key KPIs to ensure robust and accurate performance.

Key Expectations & Deliverables

- 1. Domain-Specific RAG System: Build a RAG application fine-tuned to telecom RAN tasks. This includes - Integration with telecom-specific knowledge bases, Use of public datasets like TeleQnA and O-RAN, and Support for multi-step reasoning for root cause analysis and anomaly detection.
- 2. Real-Time Feasibility: The system should be capable of processing and responding to queries in near real-time, leveraging efficient retrieval and generation techniques.
- 3. Technical Efficiency: Optimize the system to minimize resource usage (eg: RAM, GPU) while maintaining high performance. Techniques like LoRA, chunk optimization, and re-ranking can be explored.
- 4. Security & Privacy: Ensure that the system adheres to telecom industry data privacy standards and does not expose sensitive information.
- 5. Explainability: The system should provide faithful and interpretable responses, clearly indicating the sources of retrieved information and the reasoning behind generated outputs.
- 6. Final Deliverables: Working RAG application and demonstration on various use cases.

Definitive Target KPIs & Benchmarks

Your RAG application should target the following KPIs:

Metric	Target
--------	--------

Mean Reciprocal Rank (MRR)	Above 75%
Top-k Accuracy	Above 85%
Accuracy	Above 80%
Recall	Above 85%
Faithfulness	Above 90%

Suggested Open Models and Datasets

- TeleQnA: A dataset of 1,827 multiple-choice questions from 3GPP standard documents. Ideal for training and evaluating domain-specific RAG systems.
- <https://github.com/netop-team/TeleQnA>
- O-RAN Dataset: Contains real-world RAN data, including logs, alarms, and performance metrics. Can be used for anomaly detection and root cause analysis.
- Simu5G Data: Open-source data generated from a calibrated 5G simulator. useful for training models on synthetic failure scenarios.
- 3GPP Release 16 and 18 Documents: Technical specifications from 3GPP, used in the TelecomRAG framework to build knowledge bases.
- <https://github.com/netop-team>
- <https://github.com/Ali-maatouk/Tele-LLMs>

11

Real-Time Multi-User Smart Assistant for Dynamic and Noisy Smart Environments

close

Problem Statement

In today's smart homes, managing simultaneous interactions from multiple users in chaotic or noisy environments remains a challenge. The objective is to design a smart assistant that can handle multiple users' queries in real-time. This system must be capable of:

- Accurately identifying and separating different users' voices from mono or stereo audio.

- Distinguishing between commands and background noise.
- Detecting turn-taking and overlapping speech.
- Recognizing multiple keywords simultaneously.
- Delivering personalized responses based on the identified user.

The assistant must perform these tasks efficiently and effectively, even when faced with varying levels of noise, overlapping speech from up to three speakers, and diverse acoustic conditions.

Key Expectations & Deliverables

- A working model of the multi-speaker separation system capable of real-time processing.
- A showcase of the system's performance using various audio samples to illustrate its robustness in different scenarios.

Definitive Target KPIs & Benchmarks

- SI-SNR (Scale-Invariant Signal-to-Noise Ratio):
Clean audio: $>25\text{dB}$ for ≤ 2 speakers and $>15\text{dB}$ for >2 speakers.
Noisy or RIR (Room Impulse Response) audio: $>18\text{dB}$ for ≤ 2 speakers and $>10\text{dB}$ for >2 speakers.
- WER (Word Error Rate):
Clean audio: $<5\%$ for ≤ 2 speakers and $<10\%$ for >2 speakers.
Noisy or RIR audio: $<10\%$ for ≤ 2 speakers and $<20\%$ for >2 speakers.
- Model Parameters: <5 million.
- Real-Time Factor (xRT): <0.5 , ensuring real-time performance.

Suggested Open Models or Datasets

- Asteroid: An open-source toolkit for audio source separation, which includes various neural network models like DPRNN and TasNet.
- ESPNet: A speech processing toolkit that supports end-to-end speech separation and recognition models.
- LibriMix: A dataset for multi-speaker speech separation, derived from the LibriSpeech corpus, and includes 2 and 3 speaker mixtures.
- WSJ0-2mix/3mix: A dataset containing clean and noisy mixtures of speech, ideal for training and testing speech separation models.